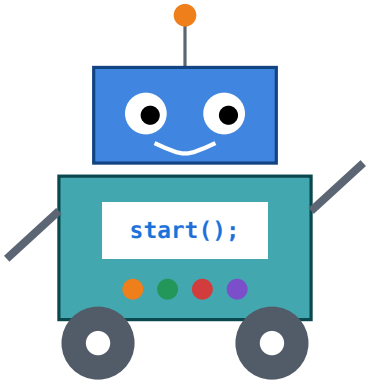
Μαθαίνω C

με το ρομπότ μου

και όλες οι εντολές του `schoolKits.h`

— εξηγημένα απλά —



Στο πρώτο μέρος μαθαίνεις τη γλώσσα **C**, βήμα βήμα, δοκιμάζοντας κάθε τι καινούργιο πάνω στο ρομπότ σου. Στο δεύτερο θα βρεις **όλες τις εντολές** που καταλαβαίνει το ρομπότ: **τι κάνει** η καθεμία, **τι γράφεις μέσα** στις παρενθέσεις και **ποιους αριθμούς** επιτρέπεται να βάλεις.

Για το ρομπότ με τον εγκέφαλο ATmega8 και το πρόγραμμα Rogic

Περιεχόμενα

Πώς είναι φτιαγμένο αυτό το βιβλίο 3

Μέρος Α: Μαθαίνω C 4

1 Τι είναι ένα πρόγραμμα; 4

2 Ο σκελετός κάθε προγράμματος 5

3 Μεταβλητές: κουτάκια για αριθμούς 5

4 Λογαριασμοί 7

5 Ερωτήσεις: `if` και `else` 8

6 Και, ή, όχι 9

7 Επανάληψη με `while` 10

8 Επανάληψη με `for` 12

9 Βγαίνω από μια επανάληψη: `break` 13

10 Οι δικές μου εντολές: συναρτήσεις 13

11 Πολλές επιλογές με `switch` 15

Μέρος Β: Το λεξικό του ρομπότ 16

12 Ξεκίνημα και τέλος 16

13 Περιμένω λίγο (χρόνος) 16

14 Μοτέρ (ρόδες) 17

15 Σέρβο (μοτέρ που πάει σε θέση) 19

16 Έξοδοι: LED, βομβητής, ήχος 20

17 Αισθητήρες (είσοδοι) 21

Μέρος Γ: Βοήθεια 23

18 Τα πιο συνηθισμένα λάθη **23**

19 Τρία ολόκληρα προγράμματα **24**

20 Πίνακας με όλα με μια ματιά **27**

21 Λύσεις των προκλήσεων **27**

Πώς είναι φτιαγμένο αυτό το βιβλίο

Το ρομπότ σου δεν καταλαβαίνει ελληνικά ούτε αγγλικά. Καταλαβαίνει μια γλώσσα προγραμματισμού που λέγεται **C**. Σε αυτό το βιβλίο θα τη μάθεις βήμα βήμα, και κάθε τι καινούργιο θα το δοκιμάζεις αμέσως πάνω στο ρομπότ σου.

- **Μέρος Α – Μαθαίνω C.** Μικρά μαθήματα, το ένα μετά το άλλο: τι είναι πρόγραμμα, μεταβλητές, ερωτήσεις, επαναλήψεις, δικές σου εντολές. Στο τέλος κάθε μαθήματος υπάρχει μια **Πρόκληση** για να δοκιμάσεις μόνος σου.
- **Μέρος Β – Το λεξικό του ρομπότ.** Όλες οι εντολές που ξέρει το ρομπότ (μοτέρ, σέρβο, LED, αισθητήρες…), η καθεμία με τη δική της κάρτα.
- **Μέρος Γ – Βοήθεια.** Τα πιο συνηθισμένα λάθη, ολόκληρα προγράμματα, ένας πίνακας με όλα με μια ματιά και οι **λύσεις** των προκλήσεων. Δοκίμασε πρώτα μόνος σου και κοίτα τη λύση μόνο αν κολλήσεις!

Πού γράφω τον κώδικα;

Στο Rogic, κάνε δεξί κλικ στο παράθυρο με τα μπλοκ και διάλεξε **«Άνοιγμα RogicEditor»**. Στη λίστα πάνω αριστερά διάλεξε **Editor**, για να μπορείς να γράφεις. Γράψε το πρόγραμμα και πάτα **Download (F8)**. Το Rogic κάνει τρία πράγματα, το ένα μετά το άλλο:

1. **ελέγχει** τον κώδικα· αν βρει λάθος, σου δείχνει σε ποια γραμμή είναι,
2. τον **μετατρέπει σε μπλοκ**, που τα βλέπεις αμέσως στο παράθυρο του Rogic,
3. **στέλνει** τα μπλοκ στο ρομπότ με το καλώδιο.

Με το κουμπί **Change to blocks** βλέπεις τα μπλοκ χωρίς να τα στείλεις.

Συμβουλή

Οι εντολές του ρομπότ είναι γραμμένες σε ένα αρχείο που λέγεται `schoolKits.h`. Είναι σαν **λεξικό**: μέσα έχει όλες τις λέξεις που καταλαβαίνει το ρομπότ. Το βρίσκεις στον φάκελο του Rogic, στον υποφάκελο `Lib`, και μπορείς να το δεις και στον επεξεργαστή κώδικα, στην καρτέλα `schoolkits.h`.

ΜΕΡΟΣ Α

Μαθαίνω C

Μικρά μαθήματα με το ρομπότ μου

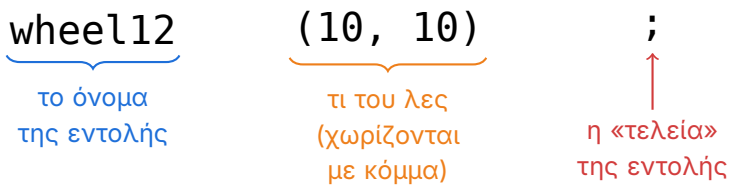
1. Τι είναι ένα πρόγραμμα;

Ένα πρόγραμμα είναι σαν μια **συνταγή μαγειρικής**: μια λίστα με βήματα. Το ρομπότ τα κάνει **ένα ένα, με τη σειρά, από πάνω προς τα κάτω**. Διαβάζει μια γραμμή, την κάνει και πάει στην επόμενη.

```
on(OUT1);      // βήμα 1: άναψε το LED της εξόδου 1
delay(1000);   // βήμα 2: περίμενε 1 δευτερόλεπτο
off(OUT1);     // βήμα 3: σβήσε το LED
```

Κάθε τέτοια γραμμή είναι μια **εντολή**. Το ρομπότ δεν σκέφτεται μόνο του: κάνει **ακριβώς** ό,τι γράψεις, ούτε περισσότερα ούτε λιγότερα. Αν ξεχάσεις το `delay(1000);`, το LED θα ανάψει και θα σβήσει τόσο γρήγορα που δεν θα το δεις καν!

Πώς γράφεται μια εντολή



- Στις **παρενθέσεις** `( )` λες στην εντολή *πόσο* ή *ποιο*. Αν δεν θέλει τίποτα, γράφεις άδειες παρενθέσεις: `start();`
- Το **ερωτηματικό** `;` μπαίνει στο τέλος κάθε εντολής, όπως η τελεία στο τέλος μιας πρότασης.
- Τα **άγκιστρα** `{ }` μαζεύουν πολλές εντολές μαζί, σαν κουτί. Θα τα δεις σε λίγο.

Σχόλια: σημειώσεις για σένα

Ό,τι γράψεις μετά από `//` μέχρι το τέλος της γραμμής είναι **σχόλιο**. Το ρομπότ το αγνοεί. Τα σχόλια είναι για να θυμάσαι εσύ (και να καταλαβαίνουν οι άλλοι) τι κάνει ο κώδικας. Για σχόλιο σε πολλές γραμμές, το ξεκινάς με `/*` και το τελειώνεις με `*/`.

```
delay(500);   // αυτό είναι σχόλιο σε μία γραμμή

/* αυτό είναι σχόλιο
   σε πολλές γραμμές */
```

Προσοχή!

- Τα **κεφαλαία και τα μικρά** μετράνε: `delay` δεν είναι το ίδιο με `Delay` ή `DELAY`.
- Γράφε τον κώδικα με το πληκτρολόγιο **στα αγγλικά**. Ελληνικά επιτρέπονται μόνο μέσα

στα σχόλια.

- Τα κενά και οι αλλαγές γραμμής δεν πειράζουν το ρομπότ. Τα βάζουμε για να διαβάζεται εύκολα ο κώδικας.

2. Ο σκελετός κάθε προγράμματος

Κάθε πρόγραμμα για το ρομπότ έχει πάντα αυτό το σχήμα. Οι δικές σου εντολές μπαίνουν στη μέση:

```c
#include "schoolkits.h"   // φέρε το λεξικό του ρομπότ

int main(void)            // εδώ ξεκινάει το πρόγραμμα
{
    start();              // ξύπνα το ρομπότ

    // ... εδώ γράφεις τις δικές σου εντολές ...

    end();                // σβήσε τα πάντα και σταμάτα
    return 0;             // «τελείωσα»
}
```

Γραμμή	Τι σημαίνει
`#include "schoolkits.h"`	Φέρνει το λεξικό με τις εντολές του ρομπότ. Χωρίς αυτό, το ρομπότ δεν ξέρει τι είναι το `delay` ή το `on`.
`int main(void)`	Εδώ ξεκινάει το πρόγραμμα. Κάθε πρόγραμμα έχει **ένα** `main`.
`{ }`	Το «κουτί» του `main`. Ό,τι είναι μέσα, ανήκει στο πρόγραμμα.
`start();`	Ξυπνάει το ρομπότ. Είναι πάντα η πρώτη εντολή.
`end();`	Σβήνει μοτέρ και εξόδους και σταματάει το πρόγραμμα.
`return 0;`	Η τελευταία γραμμή του `main`. Σημαίνει «τελείωσα».

Συμβουλή

Πρόσεξε ότι οι γραμμές μέσα στο κουτί είναι λίγο πιο μέσα (με 4 κενά). Λέγεται **εσοχή** και δείχνει με μια ματιά τι είναι μέσα σε τι. Το ρομπότ δεν τη χρειάζεται, αλλά εσύ ναι!

Πρόκληση 1

Γράψε ένα ολόκληρο πρόγραμμα (με τον σκελετό) που ανάβει το LED της `OUT1` για 2 δευτερόλεπτα και μετά το σβήνει.

3. Μεταβλητές: κουτάκια για αριθμούς

Μια **μεταβλητή** είναι ένα **κουτάκι με όνομα**. Μέσα του κρατάς έναν αριθμό, και μπορείς να τον αλλάζεις όποτε θέλεις.



ένα κουτάκι με όνομα `taxytita`
που έχει μέσα τον αριθμό `10`

```
int taxytita = 10;              // φτιάξε το κουτάκι taxytita και βάλε μέσα 10
wheel12(taxytita, taxytita);    // το ρομπότ κοιτάει τι έχει το κουτάκι: 10
```

- Το `int` λέει τι **είδος** κουτάκι φτιάχνεις: ένα κουτάκι για **ακέραιους** αριθμούς.
- Το `taxytita` είναι το **όνομα** του κουτιού. Το διαλέγεις εσύ.
- Το `=` σημαίνει «**βάλε μέσα**». Στη C **δεν** σημαίνει «ίσον»!

Αφού φτιάξεις το κουτάκι, μπορείς να του βάλεις άλλον αριθμό. Τώρα **χωρίς** το `int`, γιατί το κουτάκι υπάρχει ήδη:

```
taxytita = 15;                  // πέτα το 10 και βάλε μέσα 15
wheel12(taxytita, taxytita);    // τώρα πηγαίνει με 15
```

Γιατί να χρησιμοποιήσω μεταβλητή;

Φαντάσου ένα πρόγραμμα όπου η ταχύτητα `10` είναι γραμμένη σε 20 σημεία. Για να την αλλάξεις, πρέπει να βρεις και τα 20. Με μια μεταβλητή την αλλάζεις **σε ένα μόνο σημείο**. Και, όπως θα δεις σε λίγο, μια μεταβλητή μπορεί να **μετράει**: πόσες φορές πατήθηκε ένα κουμπί, πόσες φορές αναβόσβησε ένα LED.

Κανόνες για τα ονόματα

Σωστά ονόματα	Λάθος ονόματα
`taxytita`	`ταχύτητα` (ελληνικά γράμματα)
`xronos_strofis`	`xronos strofis` (κενό)
`fores2`	`2fores` (ξεκινάει με αριθμό)
`metritis`	`while` (λέξη της C)

Χρησιμοποίησε λατινικά γράμματα, αριθμούς και την κάτω παύλα `_`. Και θυμήσου: το `Taxytita` και το `taxytita` είναι **δύο διαφορετικά** κουτάκια.

Τι αριθμούς χωράει ένα κουτάκι

Για το ρομπότ σου γράφε πάντα `int`, και θυμήσου δύο κανόνες:

1. Μια μεταβλητή που **αλλάζει ενώ τρέχει το πρόγραμμα** (π.χ. ένας μετρητής μέσα σε επανάληψη, ή μια μεταβλητή που αλλάζει μέσα σε `if`) χωράει μόνο ακέραιους **από 0 έως 255**. Χωρίς αρνητικούς!
2. Ο χρόνος στο `delay`, η ταχύτητα των μοτέρ και η θέση του σέρβο πρέπει να είναι αριθμοί που **ξέρει το Rogic πριν τρέξει το πρόγραμμα**: σκέτος αριθμός, ή μεταβλητή που *δεν* αλλάζει μέσα σε επανάληψη ή σε `if`, όπως η `taxytita` παραπάνω.

Συμβουλή

Γιατί; Το Rogic μετατρέπει τον κώδικά σου σε **μπλοκ** για να τον στείλει στο ρομπότ, και τα μπλοκ έχουν αυτούς τους κανόνες. Αν κάτι δεν ταιριάζει, το Rogic σού το λέει και σου δείχνει τη γραμμή.

Πρόκληση 2

Φτιάξε μια μεταβλητή `xronos` με τιμή 2000. Βάλε το ρομπότ να πάει μπροστά για `xronos` χιλιοστά (με `wheel12` και `delay(xronos)`) και μετά να σταματήσει. Μετά άλλαξε μόνο τον αριθμό στη μεταβλητή και δες τι γίνεται.

4. Λογαριασμοί

Το ρομπότ είναι πολύ καλό στους λογαριασμούς. Αυτά είναι τα σύμβολα:

Σύμβολο	Πράξη	Παράδειγμα	Αποτέλεσμα
`+`	πρόσθεση	`7 + 2`	9
`-`	αφαίρεση	`7 - 2`	5
`*`	πολλαπλασιασμός	`7 * 2`	14
`/`	διαίρεση	`7 / 2`	**3** (όχι 3,5!)

Προσοχή!

Η διαίρεση με `int` **πετάει τα δεκαδικά**: το `7 / 2` κάνει 3 και το `1 / 2` κάνει `0`.

Αλλάζω μια μεταβλητή με λογαριασμό

```
int taxytita = 5;
taxytita = taxytita + 3;   // πάρε ό,τι έχει (5), πρόσθεσε 3, ξαναβάλ' το μέσα: τώρα έχει 8
wheel12(taxytita, taxytita);
```

Το `taxytita = taxytita + 3;` σε μαθηματικά θα ήταν λάθος, αλλά στη C σημαίνει: «υπολόγισε το δεξί μέρος και **βάλε** το αποτέλεσμα στο κουτάκι αριστερά».

Επειδή τέτοια πράγματα τα γράφουμε συνέχεια, υπάρχουν και συντομεύσεις:

Σύντομα	Ολόκληρο	Τι κάνει
`x++;`	`x = x + 1;`	μεγάλωσε κατά 1
`x--;`	`x = x - 1;`	μίκρυνε κατά 1
`x += 5;`	`x = x + 5;`	μεγάλωσε κατά 5
`x -= 5;`	`x = x - 5;`	μίκρυνε κατά 5

Πρόκληση 3

Φτιάξε μια μεταβλητή `misos` με τιμή `3 * 600 / 2`. Πόσο κάνει; Χρησιμοποίησέ την στο `delay(misos);` για να αναβοσβήσεις ένα LED και έλεγξε αν μάντεψες σωστά.

5. Ερωτήσεις: `if` και `else`

Μέχρι τώρα το ρομπότ έκανε πάντα τα ίδια. Με το `if` («**αν**») μπορεί να **αποφασίζει**: κάνει κάτι *μόνο αν* ισχύει μια ερώτηση.

```
if (IN1)              // ΑΝ πατήθηκε ο διακόπτης στην είσοδο 1...
{
    on(OUT1);         // ...άναψε το LED
}
```

Η ερώτηση μπαίνει μέσα στις παρενθέσεις. Αν η απάντηση είναι «ναι», γίνονται οι εντολές μέσα στα άγκιστρα. Αν είναι «όχι», το ρομπότ τις **προσπερνάει**.

`else`: αλλιώς

Με το `else` λες τι να γίνει όταν η απάντηση είναι «όχι»:

```
if (IN1)              // αν πατήθηκε...
{
    on(OUT1);         // ...άναψε
}
else                  // αλλιώς...
{
    off(OUT1);        // ...σβήσε
}
```

Ερωτήσεις με αριθμούς

Μπορείς να ρωτήσεις και για αριθμούς, με αυτά τα σύμβολα:

Σύμβολο	Ρωτάει	Παράδειγμα
`==`	είναι ίσο;	`fores == 3`
`!=`	είναι διαφορετικό;	`fores != 3`
`<`	είναι μικρότερο;	`taxytita < 10`
`>`	είναι μεγαλύτερο;	`taxytita > 10`
`<=`	είναι μικρότερο ή ίσο;	`taxytita <= 10`
`>=`	είναι μεγαλύτερο ή ίσο;	`taxytita >= 10`

`else if`: πολλές επιλογές

```
if (taxytita > 12)
{
    on(OUT3);          // πολύ γρήγορα: κόκκινο LED
}
else if (taxytita > 6)
{
    on(OUT2);          // μέτρια: κίτρινο LED
}
else
{
    on(OUT1);          // αργά: πράσινο LED
}
```

Το ρομπότ ελέγχει τις ερωτήσεις με τη σειρά και κάνει **μόνο την πρώτη** που απαντάει «ναι».

Συμβουλή

Για το ρομπότ, το `0` σημαίνει «**όχι**» και κάθε άλλος αριθμός σημαίνει «**ναι**». Γι' αυτό το `if (IN1)` δουλεύει: το `IN1` είναι `1` όταν ο διακόπτης είναι πατημένος και `0` όταν δεν είναι.

Προσοχή!

- Για να ρωτήσεις «είναι ίσο;» γράφεις **δύο** ίσον: `if (fores == 3)`. Με ένα (`if (fores = 3)`) δεν ρωτάς, αλλά **βάζεις** το 3 μέσα. Το Rogic θα σου το πει.
- **Μη** βάζεις `;` μετά την παρένθεση του `if`. Το `if (IN1);` σημαίνει «αν πατήθηκε, μην κάνεις τίποτα», και οι εντολές στα άγκιστρα γίνονται *πάντα*.

Πρόκληση 4

Φτιάξε μια μεταβλητή `taxytita`. Αν είναι μεγαλύτερη από 8, το ρομπότ να πάει μπροστά για 1 δευτερόλεπτο. Αλλιώς να ανάψει το LED της `OUT1`. Δοκίμασε με 5 και με 12.

6. Και, ή, όχι

Μερικές φορές θέλεις να κάνεις **δύο ερωτήσεις μαζί**. Γι' αυτό υπάρχουν τρία σύμβολα:

Σύμβολο	Λέξη	Η απάντηση είναι «ναι» όταν…
&&	**και**	ισχύουν **και οι δύο** ερωτήσεις
||	**ή**	ισχύει **τουλάχιστον μία** από τις δύο
!	**όχι**	**δεν** ισχύει η ερώτηση (γυρίζει το «ναι» σε «όχι» και το ανάποδο)

```
if (IN1 && IN2)      // πατήθηκαν ΚΑΙ οι δύο διακόπτες
{
    voice(OUT3);
}

if (IN1 || IN2)      // πατήθηκε ο ένας Ή ο άλλος (ή και οι δύο)
{
    on(OUT1);
}

if (!IN1)            // ΔΕΝ είναι πατημένος ο διακόπτης
{
    off(OUT1);
}
```

Συμβουλή

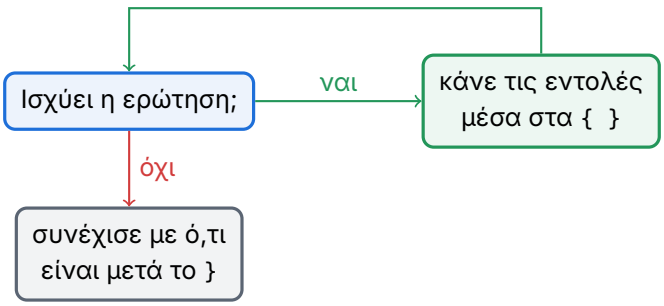
Το `||` το βρίσκεις στο πληκτρολόγιο πατώντας **Shift** μαζί με το πλήκτρο `\` (δύο φορές).

Πρόκληση 5

Βάλε δύο διακόπτες, στην `IN1` και στην `IN2`. Το LED της `OUT1` να ανάβει μόνο όταν είναι πατημένοι **και οι δύο**, και να σβήνει αλλιώς. (Θα χρειαστείς και την επανάληψη του επόμενου μαθήματος: βάλε τα πάντα μέσα σε `while (1) { ... }`.)

7. Επανάληψη με `while`

Το `while` σημαίνει «**όσο**». Κάνει τις εντολές μέσα στα άγκιστρα **ξανά και ξανά**, όσο η ερώτηση απαντάει «ναι».



Ένας μετρητής

```
int fores = 0;           // ο μετρητής ξεκινάει από το 0
while (fores < 3)        // όσο ο μετρητής είναι κάτω από 3...
{
    ledonoff(OUT1, 200); // ...αναβόσβησε το LED
    fores++;             // ...και πρόσθεσε 1 στον μετρητή
}
// εδώ το LED έχει αναβοσβήσει 3 φορές
```

Ας το παρακολουθήσουμε βήμα βήμα:

Γύρος	`fores` πριν	`fores < 3;`	Τι γίνεται
1	0	ναι	αναβοσβήνει, το `fores` γίνεται 1
2	1	ναι	αναβοσβήνει, το `fores` γίνεται 2
3	2	ναι	αναβοσβήνει, το `fores` γίνεται 3
—	3	**όχι**	βγαίνει από το `while`

`while (1)`: για πάντα

Αφού το `1` σημαίνει πάντα «ναι», το `while (1)` δεν σταματάει **ποτέ**. Το χρησιμοποιούμε όταν θέλουμε το ρομπότ να κοιτάει συνέχεια τους αισθητήρες του:

```
while (1)                // για πάντα...
{
    if (IN1)  on(OUT1);  // ...αν πατήθηκε, άναψε
    else      off(OUT1); // ...αλλιώς σβήσε
}
```

Όταν μέσα στο `if` ή στο `else` υπάρχει **μόνο μία** εντολή, μπορείς να παραλείψεις τα άγκιστρα, όπως εδώ.

Περιμένω μέχρι να γίνει κάτι

Ένα `while` με άδεια άγκιστρα απλώς **περιμένει**:

```
while (!IN1)             // όσο ΔΕΝ έχει πατηθεί ο διακόπτης...
{
}                        // ...μην κάνεις τίποτα
wheel12(10, 10);         // πατήθηκε! ξεκίνα
```

Προσοχή!

Αν στον μετρητή ξεχάσεις το `fores++;`, ο μετρητής μένει για πάντα 0, η ερώτηση απαντάει για πάντα «ναι», και το ρομπότ αναβοσβήνει μέχρι να το σβήσεις. Αυτό λέγεται **ατέρμονη επανάληψη**.

Πρόκληση 6

Με ένα `while` και έναν μετρητή, κάνε το LED της `OUT2` να αναβοσβήσει **5 φορές** και μετά το ρομπότ να παίξει τον ήχο της `OUT3` με `voice(OUT3);`.

8. Επανάληψη με `for`

Το `for` είναι ένα `while` που **μετράει μόνο του**. Το χρησιμοποιούμε όταν ξέρουμε από πριν **πόσες φορές** θέλουμε να επαναλάβουμε κάτι.

```
for (int i = 0; i < 3; i++)   // κάνε το 3 φορές
{
    ledonoff(OUT1, 200);
}
```

Κάνει ακριβώς το ίδιο με τον μετρητή του προηγούμενου μαθήματος, αλλά σε μία γραμμή. Μέσα στην παρένθεση υπάρχουν **τρία κομμάτια**, χωρισμένα με `;`:

```
for (  int i = 0;        i < 3;          i++  )
```

1. Ξεκίνα: φτιάξε μετρητή `i` και βάλε μέσα 0	2. συνέχισε όσο το `i` είναι κάτω από 3	3. μετά από κάθε γύρο, πρόσθεσε 1 στο `i`

Ο μετρητής είναι μια κανονική μεταβλητή, οπότε μπορείς να τον χρησιμοποιήσεις μέσα στην επανάληψη:

```
for (int s = 0; s <= 15; s++)   // η s πάει 0, 1, 2, ..., 15
{
    wheel12(s, s);              // το ρομπότ επιταχύνει σιγά σιγά
    delay(200);
}
wheel12(0, 0);
```

Ο μετρητής δεν χρειάζεται να ανεβαίνει κατά 1. Εδώ ένα σέρβο κάνει «σάρωση» με βήματα του 10:

```
for (int thesi = 17; thesi <= 197; thesi += 10)
{
    servo1(thesi);
    delay(100);
}
```

Συμβουλή

Όταν ο μετρητής γίνεται ταχύτητα, χρόνος ή θέση (όπως στα δύο παραδείγματα), το Rogic γράφει τα μπλοκ **γύρο γύρο**: 16 γύροι γίνονται 16 φορές τα ίδια μπλοκ. Γι' αυτό τέτοιες επαναλήψεις μπορούν να έχουν έως 32 γύρους.

Πρόκληση 7

Με ένα `for` που **μετράει ανάποδα** (από 15 μέχρι 0, με `s--`), κάνε το ρομπότ να **επιβραδύνει** σιγά σιγά μέχρι να σταματήσει.

9. Βγαίνω από μια επανάληψη: `break`

Το `break` σε βγάζει **αμέσως** έξω από το `while` ή το `for` που βρίσκεσαι, ακόμα κι αν η ερώτηση απαντάει ακόμα «ναι». Το πρόγραμμα συνεχίζει με ό,τι είναι μετά το `}`.

```
while (1)                   // για πάντα...
{
    wheel12(10, 10);        // ...πήγαινε μπροστά
    if (IN1)                // αν χτύπησες σε κάτι...
    {
        break;              // ...βγες από την επανάληψη!
    }
}
wheel12(0, 0);              // εδώ φτάνεις μόνο μετά το break
voice(OUT3);
```

Προσοχή!

Τα μπλοκ του ρομπότ έχουν **ένα** `break` σε κάθε επανάληψη, και μια επανάληψη με `break` δεν μπορεί να έχει μέσα της άλλο `while`. Αν θέλεις να βγεις για δύο λόγους, ένωσέ τους με `||`: `if (IN1 || IN2) break;`

Πρόκληση 8

Το ρομπότ να αναβοσβήνει το LED της `OUT1` **μέχρι 20 φορές** με ένα `for`, αλλά να σταματάει νωρίτερα με `break` αν πατηθεί ο διακόπτης της `IN1`.

10. Οι δικές μου εντολές: συναρτήσεις

Το καλύτερο: μπορείς να φτιάξεις **τις δικές σου εντολές**! Μια δική σου εντολή λέγεται **συνάρτηση**. Της δίνεις ένα όνομα και γράφεις τι θα κάνει. Μετά τη χρησιμοποιείς όσες φορές θέλεις, όπως τις εντολές του ρομπότ.

```c
#include "schoolkits.h"

void mprosta(int xronos)     // η δική μου εντολή «mprosta»
{
    wheel12(10, 10);         // πήγαινε μπροστά
    delay(xronos);           // για όσο χρόνο μου είπαν
    wheel12(0, 0);           // σταμάτα
}

int main(void)
{
    start();
    mprosta(1000);           // μπροστά για 1 δευτερόλεπτο
    delay(500);
    mprosta(3000);           // μπροστά για 3 δευτερόλεπτα
    end();
    return 0;
}
```

- `void` σημαίνει ότι η εντολή απλώς **κάνει κάτι**. Οι δικές σου εντολές ξεκινάνε πάντα με `void`.
- `mprosta` είναι το **όνομα**, με τους ίδιους κανόνες όπως στις μεταβλητές.
- Το `int xronos` μέσα στις παρενθέσεις είναι ένα κουτάκι που **γεμίζει** όταν χρησιμοποιείς την εντολή. Στο `mprosta(1000);` το `xronos` γίνεται 1000. Επειδή το `xronos` πηγαίνει στο `delay`, όταν χρησιμοποιείς την εντολή του δίνεις αριθμό που ξέρεις από πριν, όπως το `1000`.
- Τη συνάρτηση τη γράφεις **πάνω από** το `main`, για να την ξέρει το ρομπότ όταν τη συναντήσει.

Μια εντολή μπορεί να μη θέλει τίποτα ή να θέλει πολλά πράγματα, χωρισμένα με κόμμα:

```c
void beep(void)                       // δεν θέλει τίποτα
{
    voice(OUT3);
}

void kinisi(int aristero, int dexi, int xronos)   // θέλει τρία
{
    wheel12(aristero, dexi);
    delay(xronos);
    wheel12(0, 0);
}
```

Πρόκληση 9

Φτιάξε μια δική σου εντολή `strofi(int xronos)` που κάνει το ρομπότ να στρίψει επί τόπου για `xronos` χιλιοστά και μετά σταματάει. Χρησιμοποίησέ τη μαζί με την `mprosta` για να κάνει το ρομπότ ένα τετράγωνο (μπροστά, στροφή, 4 φορές, με ένα `for`).

11. Πολλές επιλογές με `switch`

Όταν θέλεις να ελέγξεις **μία μεταβλητή** για πολλές διαφορετικές τιμές, αντί για πολλά `else if` μπορείς να χρησιμοποιήσεις το `switch` («διακόπτης»):

```
int epilogi = 2;           // 1 = μπροστά, 2 = πίσω, κάτι άλλο = στροφή
switch (epilogi)           // τι έχει η epilogi;
{
    case 1:                // έχει 1
        wheel12(10, 10);
        break;
    case 2:                // έχει 2
        wheel12(-10, -10);
        break;
    default:               // κάτι άλλο
        wheel12(10, -10);
        break;
}
delay(1000);
wheel12(0, 0);
```

- Κάθε `case` έχει έναν αριθμό και `:` στο τέλος.
- Κάθε `case` τελειώνει με `break;`. Αν το ξεχάσεις, το Rogic θα σου το πει.
- Το `default` γίνεται όταν δεν ταιριάζει κανένα `case`.

Τώρα ξέρεις αρκετή C για να κάνεις σχεδόν τα πάντα με το ρομπότ σου. Στο Μέρος Β θα βρεις όλες τις εντολές του.

Οι λέξεις της C που έμαθες

`int`	κουτάκι για ακέραιους	`while`	επανάλαβε όσο…
`char`	μικρό κουτάκι (0–255)	`for`	επανάλαβε τόσες φορές
`if`	αν…	`break`	βγες από την επανάληψη
`else`	αλλιώς…	`void`	η αρχή μιας δικής σου εντολής
`switch`	διάλεξε ανάλογα με έναν αριθμό	`return`	η τελευταία γραμμή του `main`
`case`	μία επιλογή του `switch`	`default`	όλες οι άλλες επιλογές

ΜΕΡΟΣ Β

Το λεξικό του ρομπότ

Όλες οι εντολές που ξέρει το ρομπότ σου

Τώρα που ξέρεις C, ήρθε η ώρα να γνωρίσεις όλες τις εντολές του ρομπότ. Δεν χρειάζεται να τις μάθεις απ' έξω: όταν θέλεις να κάνεις κάτι, βρες την κάρτα της εντολής και δες πώς γράφεται.

Πώς διαβάζεις τις κάρτες

Κάθε εντολή έχει τη δική της χρωματιστή κάρτα. Σε κάθε κάρτα θα δεις:

- **Τι κάνει** η εντολή.
- **Τι γράφεις μέσα στις παρενθέσεις** και ποιους αριθμούς επιτρέπεται να βάλεις.
- **Τι σου δίνει πίσω**, για τους αισθητήρες, που «απαντάνε» με ναι ή όχι.
- **Παράδειγμα** που μπορείς να αντιγράψεις.

Θυμήσου τους δύο κανόνες από το μάθημα των μεταβλητών: οι αριθμοί που βάζεις στις εντολές πρέπει να είναι γνωστοί πριν τρέξει το πρόγραμμα, και κάθε κάρτα γράφει από πόσο μέχρι πόσο μπορείς να βάλεις.

12. Ξεκίνημα και τέλος

`start();`

Τι κάνει: Ξυπνάει το ρομπότ: ετοιμάζει τους αισθητήρες, τις εξόδους και τα μοτέρ.
Τι γράφεις μέσα στις παρενθέσεις: Τίποτα. Γράφεις άδειες παρενθέσεις.

Προσοχή!

Πρέπει να είναι η **πρώτη** εντολή του προγράμματος. Χωρίς αυτήν, τίποτα δεν δουλεύει σωστά.

`end();`

Τι κάνει: Σβήνει **όλα** τα μοτέρ και όλες τις εξόδους και μετά σταματάει το πρόγραμμα για πάντα.
Τι γράφεις μέσα στις παρενθέσεις: Τίποτα.

Προσοχή!

Ό,τι γράψεις **μετά** το `end();` δεν θα γίνει ποτέ. Για να ξαναξεκινήσει το πρόγραμμα, σβήνεις και ανάβεις το ρομπότ.

13. Περιμένω λίγο (χρόνος)

Οι εντολές εκτελούνται πάρα πολύ γρήγορα, η μία μετά την άλλη. Αν θέλεις το ρομπότ να κάνει κάτι *για λίγη ώρα* (π.χ. να πάει μπροστά για 2 δευτερόλεπτα), του λες να **περιμένει**. Όσο

περιμένει, τα μοτέρ και τα LED μένουν όπως τα άφησες.

`delay(χιλιοστά);`

Τι κάνει: Το ρομπότ περιμένει όσο χρόνο του πεις. Είναι η εντολή που χρησιμοποιεί και το Rogic.
Τι γράφεις μέσα στις παρενθέσεις: Τον χρόνο σε **χιλιοστά του δευτερολέπτου**. Το `1000` είναι 1 δευτερόλεπτο.

`delay(100);`	δέκατο του δευτερολέπτου (το πιο μικρό)
`delay(500);`	μισό δευτερόλεπτο
`delay(1000);`	1 δευτερόλεπτο
`delay(2500);`	2,5 δευτερόλεπτα
`delay(25500);`	25,5 δευτερόλεπτα

Προσοχή!

Η `delay` μετράει μόνο σε **βήματα του 100**. Το `delay(250)` περιμένει 0,2 δευτερόλεπτα (κόβει το 50). Το `delay(50)` **δεν περιμένει καθόλου**!

`sleep(δέκατα);`

Τι κάνει: Το ίδιο με την `delay`, αλλά μετράει σε **δέκατα του δευτερολέπτου**.
Τι γράφεις μέσα στις παρενθέσεις: Έναν αριθμό από `0` έως `255`. Το `10` είναι 1 δευτερόλεπτο, το `255` είναι 25,5 δευτερόλεπτα.
Παράδειγμα:

```
sleep(20);    // περίμενε 2 δευτερόλεπτα
```

14. Μοτέρ (ρόδες)

Το ρομπότ μπορεί να έχει έως **4 μοτέρ**. Συνήθως το **μοτέρ 1** και το **μοτέρ 2** είναι οι δύο ρόδες. Σε κάθε μοτέρ λες δύο πράγματα:

- **Πόσο γρήγορα**: από `0` (σταματημένο) έως `15` (τέρμα γκάζι).
- **Προς τα πού**: με το πρόσημο. Θετικός αριθμός (π.χ. `10`) = μπροστά, αρνητικός (π.χ. `-10`) = πίσω.

Συμβουλή

Το «μπροστά» εδώ είναι το ίδιο με το «μπροστά» του Rogic. Αν το ρομπότ σου πηγαίνει ανάποδα, τα καλώδια του μοτέρ είναι συνδεδεμένα αντίστροφα: γύρισέ τα ή άλλαξε το πρόσημο.

`motor1(ταχύτητα); · motor2(...); · motor3(...); · motor4(...);`

Τι κάνει: Βάζει ένα μοτέρ να γυρίσει. Το `motor1` είναι το μοτέρ 1, το `motor2` το μοτέρ 2 κ.ο.κ. Το μοτέρ συνεχίζει να γυρίζει μέχρι να του πεις κάτι άλλο.
Τι γράφεις μέσα στις παρενθέσεις: Την ταχύτητα, από `-15` έως 15.

`motor1(15);`	μπροστά, όσο πιο γρήγορα γίνεται
`motor1(8);`	μπροστά, με τη μισή περίπου ταχύτητα
`motor1(0);`	σταμάτα
`motor1(-8);`	πίσω, με τη μισή περίπου ταχύτητα
`motor1(-15);`	πίσω, όσο πιο γρήγορα γίνεται

Αν γράψεις πάνω από 15 (π.χ. 50), το ρομπότ το κάνει μόνο του 15. Το ίδιο και κάτω από −15.

Προσοχή!

Με πολύ μικρές ταχύτητες (1, 2, 3) το μοτέρ μπορεί να μην έχει αρκετή δύναμη να κουνήσει το ρομπότ. Ξεκίνα από το 6 και πάνω.

`wheel12(αριστερό, δεξί);`

Τι κάνει: Δίνει ταχύτητα **και στις δύο ρόδες μαζί**: το πρώτο νούμερο πάει στο μοτέρ 1 και το δεύτερο στο μοτέρ 2. Είναι το ίδιο με το να γράψεις `motor1(...)` και μετά `motor2(...)`.
Τι γράφεις μέσα στις παρενθέσεις: Δύο ταχύτητες, η καθεμία από `-15` έως 15.

`wheel12(10, 10);`	μπροστά
`wheel12(-10, -10);`	πίσω
`wheel12(10, -10);`	στροφή επί τόπου προς τη μία μεριά
`wheel12(-10, 10);`	στροφή επί τόπου προς την άλλη μεριά
`wheel12(10, 4);`	στροφή «φαρδιά», ενώ προχωράει
`wheel12(0, 0);`	σταμάτα

Παράδειγμα:

```
wheel12(10, 10);  // πήγαινε μπροστά...
delay(2000);      // ...για 2 δευτερόλεπτα
wheel12(0, 0);    // σταμάτα
```

`wheel34(μοτέρ3, μοτέρ4);`

Τι κάνει: Το ίδιο με την `wheel12`, αλλά για το μοτέρ 3 και το μοτέρ 4.
Τι γράφεις μέσα στις παρενθέσεις: Δύο ταχύτητες από `-15` έως 15.

`motor(θύρα, κατεύθυνση, ταχύτητα);`

Τι κάνει: Βάζει ένα μοτέρ να γυρίσει, γράφοντας χωριστά την κατεύθυνση και την ταχύτητα. Αυτή την εντολή γράφει το Rogic όταν μετατρέπει τα μπλοκ σε κώδικα.

Τι γράφεις μέσα στις παρενθέσεις: Τρία πράγματα:

- **θύρα**: `0` για το μοτέρ 1, `2` για το μοτέρ 2, `4` για το μοτέρ 3, `6` για το μοτέρ 4.
- **κατεύθυνση**: `DIR_CW` για μπροστά ή `DIR_CCW` για πίσω.
- **ταχύτητα**: από `0` έως `15` (εδώ *χωρίς* μείον).

Παράδειγμα:

```
motor(0, DIR_CW, 12);   // μοτέρ 1 μπροστά με ταχύτητα 12
motor(2, DIR_CCW, 12);  // μοτέρ 2 πίσω με ταχύτητα 12
```

Προσοχή!

Η θύρα είναι `0, 2, 4, 6` και **όχι** 1, 2, 3, 4! Και η ταχύτητα εδώ είναι μόνο από `0` έως `15`. Γι' αυτό είναι πιο εύκολο να χρησιμοποιείς τις `motor1`–`motor4`.

Προσοχή!

Τα **μοτέρ 3 και 4** χρησιμοποιούν τις ίδιες υποδοχές με τις **εξόδους 1–4**: το μοτέρ 3 πιάνει τις `OUT1` και `OUT2` και το μοτέρ 4 τις `OUT3` και `OUT4`. Αν βάλεις μοτέρ 3, μη βάλεις LED ή σέρβο στις εξόδους 1 και 2.

15. Σέρβο (μοτέρ που πάει σε θέση)

Ένα **σέρβο** δεν γυρίζει συνέχεια όπως οι ρόδες. Πηγαίνει σε μια **θέση** και μένει εκεί, σαν δείκτης ρολογιού. Είναι καλό για χέρια, δαγκάνες και πόρτες.

servo1(θέση); · servo2(...); · servo3(...); · servo4(...); · servo5(...);

Τι κάνει: Πηγαίνει το σέρβο σε μια θέση. Το `servo1` είναι το σέρβο στην έξοδο `OUT1`, το `servo2` στην `OUT2`, και έτσι μέχρι το `servo5` στην `OUT5`. Το σέρβο μένει στη θέση του μέχρι να του πεις άλλη.

Τι γράφεις μέσα στις παρενθέσεις: Τη θέση, έναν αριθμό από `0` έως `210`. Στο Rogic, ο αριθμός είναι περίπου «μοίρες + 17»:

`servo1(17);`	η μία άκρη (περίπου 0 μοίρες)
`servo1(62);`	περίπου 45 μοίρες
`servo1(107);`	η μέση (περίπου 90 μοίρες)
`servo1(152);`	περίπου 135 μοίρες
`servo1(197);`	η άλλη άκρη (περίπου 180 μοίρες)

Αν γράψεις πάνω από 210, το ρομπότ το κάνει μόνο του 210.

Παράδειγμα:

```
servo1(17);    // άνοιξε τη δαγκάνα
delay(1000);   // άφησε χρόνο στο σέρβο να φτάσει
servo1(107);   // κλείσε τη δαγκάνα στη μέση
```

Συμβουλή

Το σέρβο θέλει λίγο χρόνο για να φτάσει στη θέση. Βάλε ένα `delay` μετά από κάθε κίνηση. Και κάθε σέρβο είναι λίγο διαφορετικό: δοκίμασε για να βρεις τους δικούς σου αριθμούς.

Προσοχή!

Αν μια έξοδος έγινε σέρβο, το `on()` σε αυτή την έξοδο δεν κάνει πια τίποτα, και το `off()` κάνει το σέρβο να «χαλαρώσει» και να μην κρατάει τη θέση του.

16. Έξοδοι: LED, βομβητής, ήχος

Το ρομπότ έχει **5 εξόδους**, με τα ονόματα `OUT1` έως `OUT5`. Εκεί βάζεις πράγματα που τα ανάβεις και τα σβήνεις: LED, βομβητή, μονάδα ήχου, μικρά μοτεράκια.

on(έξοδος); · off(έξοδος);

Τι κάνει: Το `on` **ανάβει** μια έξοδο με όλη της τη δύναμη και το `off` τη **σβήνει**. Η έξοδος μένει έτσι μέχρι να της πεις κάτι άλλο.

Τι γράφεις μέσα στις παρενθέσεις: Ποια έξοδο: `OUT1`, `OUT2`, `OUT3`, `OUT4` ή `OUT5` (μπορείς να γράψεις και σκέτο 1 έως 5).

Παράδειγμα:

```
on(OUT1);      // άναψε το LED της εξόδου 1
delay(1000);
off(OUT1);     // σβήσ' το
```

`voice(έξοδος);`

Τι κάνει: Παίζει τον ήχο της **μονάδας ήχου** που είναι συνδεδεμένη σε αυτή την έξοδο. Ανάβει την έξοδο για 0,2 δευτερόλεπτα, που είναι το σήμα για να ξεκινήσει ο ήχος, και μετά τη σβήνει.
Τι γράφεις μέσα στις παρενθέσεις: Ποια έξοδο: `OUT1` έως `OUT5`.
Παράδειγμα:

```
voice(OUT3);   // παίξε τον ήχο της μονάδας στην έξοδο 3
```

Συμβουλή

Η `voice` περιμένει 0,2 δευτερόλεπτα μόνη της, άρα το πρόγραμμα σταματάει για λίγο εκεί.

`ledonoff(έξοδος, χιλιοστά);`

Τι κάνει: Αναβοσβήνει μια έξοδο **μία φορά**: την ανάβει, περιμένει, τη σβήνει και περιμένει ξανά τον ίδιο χρόνο.
Τι γράφεις μέσα στις παρενθέσεις: Δύο πράγματα: την έξοδο (`OUT1` έως `OUT5`) και πόσο να μείνει αναμμένη, σε χιλιοστά του δευτερολέπτου, σε βήματα του 100 (όπως στη `delay`).
Παράδειγμα:

```
while (1)
{
    ledonoff(OUT2, 200);   // αναβόσβημα για πάντα
}
```

`setSegment(αριθμός);`

Τι κάνει: Δείχνει έναν αριθμό από 0 έως 15 με τις εξόδους `OUT1`–`OUT4`, για μια οθονίτσα αριθμών (ψηφίο 7 τμημάτων) που μετατρέπει τις 4 εξόδους σε νούμερο. Κάθε έξοδος έχει μια «αξία»: `OUT1` = 1, `OUT2` = 2, `OUT3` = 4, `OUT4` = 8. Ανάβουν οι έξοδοι που αθροίζουν τον αριθμό σου.
Τι γράφεις μέσα στις παρενθέσεις: Έναν αριθμό από `0` έως `15`.
Παράδειγμα:

```
setSegment(5);    // 5 = 1 + 4: ανάβουν η OUT1 και η OUT3
```

17. Αισθητήρες (είσοδοι)

Το ρομπότ έχει **7 εισόδους**, τις `IN1` έως `IN7`. Εκεί βάζεις αισθητήρες: διακόπτη επαφής, αισθητήρα υπερύθρων, μαγνητικό, κίνησης, κλίσης. Οι είσοδοι δεν είναι εντολές: είναι **ερωτήσεις** που απαντάνε «ναι» ή «όχι».

IN1 · IN2 · IN3 · IN4 · IN5 · IN6 · IN7

Τι κάνει: Ρωτάει τον αισθητήρα της εισόδου αν **νιώθει κάτι** (πατήθηκε ο διακόπτης, ο αισθητήρας βλέπει κάτι).
Τι γράφεις μέσα στις παρενθέσεις: Τίποτα, ούτε καν παρενθέσεις. Γράφεις σκέτο `IN1`. Μπορείς να γράψεις και με μικρά: `in1`.
Τι σου δίνει πίσω: `1` (ναι) όταν ο αισθητήρας νιώθει κάτι, `0` (όχι) όταν δεν νιώθει τίποτα. Τις βάζουμε μέσα σε `if` («αν») ή `while` («όσο»).
Παράδειγμα:

```
if (IN1)              // αν πατήθηκε ο διακόπτης στην είσοδο 1...
{
    on(OUT1);         // ...άναψε το LED
}
else                  // αλλιώς...
{
    off(OUT1);        // ...σβήσ' το
}
```

Το `!` σημαίνει «**όχι**». Έτσι μπορείς να περιμένεις μέχρι να πατηθεί κάτι:

```
while (!IN1)          // όσο ΔΕΝ έχει πατηθεί ο διακόπτης...
{
}                     // ...μην κάνεις τίποτα, απλώς περίμενε
wheel12(10, 10);      // πατήθηκε! ξεκίνα
```

getIn(θέση)

Τι κάνει: Διαβάζει μια είσοδο «ανάποδα», όπως τη βλέπει το ηλεκτρονικό κύκλωμα.
Τι γράφεις μέσα στις παρενθέσεις: Θέση από `0` έως 6: το `0` είναι η `IN1`, ..., το 6 είναι η `IN7`.
Τι σου δίνει πίσω: `1` όταν ο αισθητήρας **δεν** νιώθει τίποτα και `0` όταν νιώθει.

Συμβουλή

Είναι μπερδεμένο γιατί είναι ανάποδο και μετράει από το 0. Χρησιμοποίησε καλύτερα τα `IN1`–`IN7`.

ΜΕΡΟΣ Γ

Βοήθεια

Λάθη, ολόκληρα προγράμματα, πίνακας και λύσεις

18. Τα πιο συνηθισμένα λάθη

1. **Ξέχασα το `;`** στο τέλος μιας εντολής. Το Rogic θα βγάλει λάθος στην επόμενη γραμμή.
2. **Ξέχασα ένα άγκιστρο `}`.** Κάθε `{` θέλει το δικό του `}`. Οι εσοχές σε βοηθάνε να δεις ποιο λείπει.
3. **Έγραψα `=` αντί για `==` μέσα σε `if`.** Το `if (x = 3)` δεν ρωτάει, αλλά βάζει το 3 στο `x`. Γράψε `if (x == 3)`.
4. **Έβαλα `;` μετά το `if (...)` ή το `while (...)`.** Τότε το `if` ή το `while` δεν ελέγχει τίποτα από όσα είναι στα άγκιστρα.
5. **Το πρόγραμμα κόλλησε σε μια επανάληψη.** Μήπως ξέχασες να αλλάξεις τον μετρητή, π.χ. με `fores++;`
6. **Η διαίρεση βγάζει λάθος αποτέλεσμα.** Με ακέραιους, το `7 / 2` κάνει `3`: τα δεκαδικά πετιούνται.
7. **Έγραψα την ίδια μεταβλητή με `int` δύο φορές.** Το `int` μπαίνει μόνο την πρώτη φορά, όταν φτιάχνεις το κουτάκι.
8. **Έγραψα `Delay` αντί για `delay`.** Τα κεφαλαία και τα μικρά μετράνε.
9. **Έγραψα `delay(50)` και δεν περιμένει.** Η `delay` μετράει σε βήματα του 100: γράψε τουλάχιστον `delay(100)`.
10. **Το ρομπότ προχωράει για πάντα.** Ξέχασες να σταματήσεις τα μοτέρ: `wheel12(0, 0);`
11. **Το ρομπότ δεν κάνει τίποτα μετά από ένα σημείο.** Μήπως έβαλες `end();` πιο πάνω; Μετά το `end()` δεν εκτελείται τίποτα.
12. **Το μοτέρ βουίζει αλλά δεν κινείται.** Η ταχύτητα είναι πολύ μικρή: δοκίμασε `6` και πάνω.
13. **Έβαλα `motor(1, DIR_CW, 10)` και κάνει περίεργα.** Στη `motor` η θύρα είναι `0, 2, 4, 6`.
14. **Το ρομπότ γυρίζει γύρω γύρω αντί να πάει ευθεία.** Το ένα μοτέρ είναι συνδεδεμένο ανάποδα: δοκίμασε `wheel12(10, -10)` ή γύρισε τα καλώδια.
15. **Το LED στην `OUT1` δεν ανάβει.** Μήπως χρησιμοποιείς το μοτέρ 3 ή το `servo1`; Πιάνουν την ίδια έξοδο.
16. **Το Rogic λέει ότι κάτι «δεν μπορεί να γίνει μπλοκ».** Διάβασε το μήνυμα: γράφει τη γραμμή και τι να αλλάξεις. Συνήθως είναι ένας από τους δύο κανόνες των μεταβλητών (από 0 έως 255, και αριθμοί που είναι γνωστοί πριν τρέξει το πρόγραμμα).
17. **Το Rogic λέει ότι το πρόγραμμα είναι πολύ μεγάλο.** Στο ρομπότ χωράνε λίγα μπλοκ. Βάλε ό,τι επαναλαμβάνεται σε μια επανάληψη ή σε μια δική σου εντολή, αντί να το γράφεις πολλές φορές.

18. **Βγαίνει λάθος σε μια γραμμή που μοιάζει σωστή.** Μήπως γράφτηκε κάτι με ελληνικούς χαρακτήρες έξω από σχόλιο; Γύρισε το πληκτρολόγιο στα αγγλικά και ξαναγράψ' το.

19. Τρία ολόκληρα προγράμματα

1. Χορευτικό

```
#include "schoolkits.h"

int main(void)
{
    start();
    wheel12(10, 10);   delay(1500);   // μπροστά 1,5 δευτ.
    wheel12(10, -10);  delay(1000);   // στροφή επί τόπου
    wheel12(-10, -10); delay(1500);   // πίσω 1,5 δευτ.
    wheel12(0, 0);                    // σταμάτα
    voice(OUT3);                      // και πες κάτι!
    end();
    return 0;
}
```

2. Ρομπότ που αποφεύγει τα εμπόδια

Διακόπτης επαφής μπροστά, στην `IN1`.

```c
#include "schoolkits.h"

int main(void)
{
    start();
    while (1)
    {
        if (IN1)                        // χτύπησε σε κάτι!
        {
            wheel12(-10, -10);          // κάνε πίσω
            delay(800);
            wheel12(10, -10);           // στρίψε
            delay(600);
        }
        else
        {
            wheel12(10, 10);            // αλλιώς πήγαινε μπροστά
        }
    }
    end();
    return 0;
}
```

3. Μετράω τα χτυπήματα

Διακόπτης επαφής μπροστά, στην `IN1`. Το ρομπότ σταματάει μετά από 3 χτυπήματα.

```c
#include "schoolkits.h"

int main(void)
{
    start();
    int xtypimata = 0;              // πόσες φορές χτύπησε
    while (xtypimata < 3)           // μέχρι να χτυπήσει 3 φορές
    {
        wheel12(10, 10);            // μπροστά
        if (IN1)                    // χτύπησε!
        {
            xtypimata++;            // μέτρα το χτύπημα
            wheel12(-10, -10);      // κάνε πίσω
            delay(500);
            wheel12(10, -10);       // στρίψε
            delay(400);
        }
    }
    wheel12(0, 0);                  // σταμάτα
    voice(OUT3);                    // τέλος!
    end();
    return 0;
}
```

20. Πίνακας με όλα με μια ματιά

Εντολή	Τι κάνει και τι βάζεις μέσα
`start();`	ξύπνα το ρομπότ (πρώτη εντολή)
`end();`	σβήσε τα πάντα και σταμάτα για πάντα
`delay(χιλιοστά);`	περίμενε· σε βήματα του 100 (1000 = 1 δευτερόλεπτο)
`sleep(δέκατα);`	περίμενε· 0 έως 255 δέκατα
`motor1(ταχ.); ... motor4(ταχ.);`	ένα μοτέρ· −15 (πίσω) έως 15 (μπροστά), 0 = στάση
`wheel12(αρ., δεξ.);`	μοτέρ 1 και 2 μαζί· −15 έως 15 το καθένα
`wheel34(μ3, μ4);`	μοτέρ 3 και 4 μαζί· −15 έως 15 το καθένα
`motor(θύρα, κατ., ταχ.);`	θύρα 0/2/4/6 · `DIR_CW` / `DIR_CCW` · ταχ. 0 έως 15
`servo1(θέση); ... servo5(θέση);`	σέρβο στην OUT1–OUT5· 0 έως 210 (μέση: 107)
`on(OUTx); off(OUTx);`	άναψε / σβήσε έξοδο· OUT1 έως OUT5
`voice(OUTx);`	παίξε τον ήχο της μονάδας ήχου
`ledonoff(OUTx, χιλ.);`	αναβόσβησε μία φορά· χιλιοστά σε βήματα του 100
`setSegment(αριθμός);`	αριθμός 0 έως 15 στις OUT1–OUT4
`IN1 ... IN7`	ρωτάει αισθητήρα· 1 = νιώθει κάτι
`getIn(θέση)`	αισθητήρας «ανάποδα»· θέση 0 έως 6 (0 = IN1)

21. Λύσεις των προκλήσεων

Υπάρχουν πολλοί σωστοί τρόποι για να λύσεις κάθε πρόκληση. Αν το δικό σου πρόγραμμα είναι διαφορετικό αλλά το ρομπότ κάνει αυτό που ζητήθηκε, **είναι κι αυτό σωστό!** Για να χωράνε, γράφουμε μόνο ό,τι μπαίνει ανάμεσα στο `start();` και στο `end();`, εκτός αν χρειάζεται κάτι πάνω από το `main`.

Πρόκληση 1: το πρώτο μου πρόγραμμα

```c
#include "schoolkits.h"

int main(void)
{
    start();
    on(OUT1);        // άναψε
    delay(2000);     // περίμενε 2 δευτερόλεπτα
    off(OUT1);       // σβήσε
    end();
    return 0;
}
```

Πρόκληση 2: μεταβλητή για τον χρόνο

```
int xronos = 2000;   // άλλαξε μόνο αυτόν τον αριθμό
wheel12(10, 10);
delay(xronos);
wheel12(0, 0);
```

Πρόκληση 3: λογαριασμός

Το `3 * 600 / 2` κάνει πρώτα `3 * 600 = 1800` και μετά `1800 / 2 = 900`.

```
int misos = 3 * 600 / 2;   // 900
on(OUT1);
delay(misos);              // 0,9 δευτερόλεπτα
off(OUT1);
```

Πρόκληση 4: `if` και `else`

```
int taxytita = 12;         // δοκίμασε και με 5
if (taxytita > 8)
{
    wheel12(taxytita, taxytita);
    delay(1000);
    wheel12(0, 0);
}
else
{
    on(OUT1);
    delay(1000);
}
```

Πρόκληση 5: και οι δύο διακόπτες

```
while (1)
{
    if (IN1 && IN2)  on(OUT1);   // και οι δύο πατημένοι
    else             off(OUT1);  // αλλιώς σβηστό
}
```

Πρόκληση 6: πέντε αναβοσβησίματα με `while`

```
int fores = 0;
while (fores < 5)
{
    ledonoff(OUT2, 200);
    fores++;
}
voice(OUT3);
```

Πρόκληση 7: επιβράδυνση με `for`

```
for (int s = 15; s >= 0; s--)   // η s πάει 15, 14, ..., 0
{
    wheel12(s, s);
    delay(200);
}
```

Πρόκληση 8: `for` με `break`

```
for (int i = 0; i < 20; i++)
{
    if (IN1)               // πατήθηκε ο διακόπτης;
    {
        break;             // σταμάτα νωρίτερα
    }
    ledonoff(OUT1, 200);
}
```

Πρόκληση 9: τετράγωνο με δικές μου εντολές

```c
#include "schoolkits.h"

void mprosta(int xronos)
{
    wheel12(10, 10);
    delay(xronos);
    wheel12(0, 0);
}

void strofi(int xronos)
{
    wheel12(10, -10);      // στροφή επί τόπου
    delay(xronos);
    wheel12(0, 0);
}

int main(void)
{
    start();
    for (int i = 0; i < 4; i++)   // 4 πλευρές
    {
        mprosta(1500);
        strofi(600);       // άλλαξε το 600 μέχρι η στροφή να βγαίνει ίσια
    }
    end();
    return 0;
}
```